The Image Lives in Two Memories: Visual Information in Hybrid Linear-Attention Vision-Language Models

Minseok Shin
Horcrux Technologies, Seoul, Republic of Korea
https://www.horcruxtechnologies.com
horcruxtech.ai@gmail.com stevenshin16@korea.ac.kr

Abstract. Hybrid vision-language models that interleave linear-attention layers with softmax attention read an image into two memories at once: the key-value (KV) rows of the visual tokens in the attention layers, and a fixed-size recurrent state in the linear-attention layers that has absorbed every visual token. Visual-token pruning and multimodal KV-cache compression assume that the image lives in the first of these. We ask where it actually lives. On Qwen3.5 (Gated DeltaNet hybrids, 4B and 9B, early fusion) we alter one memory exactly once, right after the last image token, and let the question and the answer read the altered memory. A dataset-level ratio of teacher-forced answer losses, anchored at a no-image run and an unaltered run, measures how much of the image's contribution each memory retains, with paired bootstrap intervals, on DocVQA, ChartQA, TextVQA and POPE. [TBD: one sentence per finding: (i) KV zero vs state zero retained, per task family; (ii) bits needed by each memory after reading, with and without error feedback; (iii) fewer visual tokens versus fewer bits at equal memory; (iv) whether 9B agrees.]

Keywords: vision-language models; hybrid linear attention; Gated DeltaNet; KV cache; recurrent state; visual token pruning; quantization; interpretability.

1 Introduction

A transformer vision-language model (VLM) keeps everything it has read about an image in one place: the keys and values of the visual tokens, one row per token and layer. Visual-token pruning [1–5] and multimodal KV-cache compression [6, 7] are built on that picture: removing or compressing a visual token's row removes or compresses what the model knows about that part of the image.

Hybrid models break the picture. Qwen3.5 [8] repeats three Gated DeltaNet (GDN) layers [9] and one gated softmax-attention layer, and it is natively multimodal: visual features enter at the input embedding and pass through the same stack as text (early fusion; Section 3.1). When the last image token has been processed, the image has been written into two memories. The attention layers hold the KV rows of the visual tokens, as in a transformer. The GDN layers hold a fixed-size recurrent state that has been updated by every visual token and is not indexed by token at all. In our setting this state is larger than the visual KV rows of any image we feed (Section 3.3).

Which of the two memories does the model consult when it answers a question about the image? The answer decides what token pruning and KV compression can safely do to a hybrid VLM, and whether the recurrent state, which a serving engine stores at full precision regardless of the KV format [10], is a place where image information can be lost. It is not obvious a priori. The state is written by every visual token but has a finite capacity and forgets through a learned decay; the KV rows keep every token exactly but sit in only one layer in four.

We answer the question with training-free interventions (Figure 1). The prompt is read in two stages: the prefix and the image first, then the question (Section 4.1). Between the stages we alter exactly one memory once: we zero it, quantize it, or replace it by matched noise, or we remove visual tokens before they are read. The question and the answer then read the altered memory. We measure the effect with the teacher-forced loss of the reference answer, normalised between a run without the image and an unaltered run (Section 4.3), so that 1 means the altered memory still supplies everything the image contributed and 0 means nothing is left.

Two text-only studies of the same model family frame the question. Split-prefill experiments on Qwen3.5 and Falcon-H1 keep only the KV cache or only the recurrent state of a prefilled context and find that exact retrieval survives only through attention, while the output language and a persona survive through the recurrent state [11]. Quantizing both memories at every write found the recurrent state to be the fragile one under long generation [12]. Neither involves images. An image differs from a text context in two ways that matter here: it is long (up to 1,280 tokens in our setting) and dense, and the questions asked about it range from reading a single word (DocVQA, TextVQA) to judging whether an object is present at all (POPE). The retrieval-versus-gist split of [11] predicts that reading tasks depend on the KV rows and presence judgements less so; our tasks test that prediction for visual content. For hybrid Mamba–Transformer video models, token reduction has been found to behave like compression rather than dropping because the state keeps information from discarded tokens [13]; we measure that state contribution directly, for still images and Gated DeltaNet.

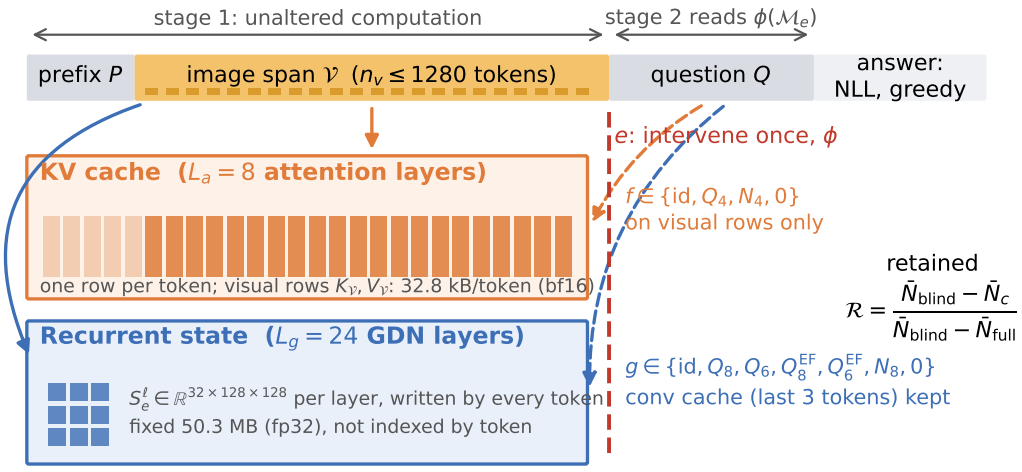


Figure 1: Overview. The prompt is read in two stages. After the last image token (e) the caches hold the image in two memories: the KV rows of the visual tokens in the $L_a = 8$ attention layers, and the recurrent state of the $L_g = 24$ GDN layers, which every visual token has updated. One memory is altered once (f or g), the question then reads the altered memory, and the reference answer's teacher-forced loss gives the retained ratio $\mathcal{R}$. Sizes for Qwen3.5-4B/9B.

Contributions. [TBD: Final bullets after phases 1–4. Planned:]

- A one-shot intervention protocol for hybrid VLMs that separates the two memories at the end of the image span, with a dataset-level retained ratio and paired bootstrap intervals (Sections 4.1–4.3).
- [TBD: Phase 1: which memory carries the image, per task (zero and noise interventions).]
- [TBD: Phase 2: bits each memory needs after reading; error feedback during the image prefill; one-shot versus every-write quantization.]
- [TBD: Phase 3: visual-token deletion versus KV quantization at equal memory.]
- [TBD: Phase 4: replication at 9B.]

2 Related Work

Visual-token reduction. FastV drops visual tokens after an early layer on the evidence of low attention to them [1]; SparseVLM ranks visual tokens with text-guided attention [2]; LLaVA-PruMerge and VisionZip select and merge tokens at the vision encoder output [3, 4]; PyramidDrop removes tokens progressively with depth [5]; token merging goes back to ToMe [14]. All of these are designed and evaluated on transformer language backbones, where the visual-token rows are the only memory of the image. For Mamba-based multimodal models, DTP scores visual tokens from the selective-scan parameters instead of attention [15]; for hybrid video models, token reduction is reported to act as compression because the state keeps what dropped tokens contributed [13].

Multimodal KV-cache compression. LOOK-M and VL-Cache compress the KV cache of multimodal inputs with modality-aware eviction and budget allocation [6, 7]. KV quantization for text, e.g. KIVI [16] and KVQuant [17], applies unchanged to the visual rows we quantize.

Where visual information flows. Attention-knockout and patching studies of transformer VLMs trace how image information reaches the answer. In InternVL2-76B and LLaVA-1.5-7B, the query tokens come to store global image information, cross-modal flow is concentrated in about a quarter of the layers in the middle of the network, and fine-grained details are read from the image tokens directly [18]. In LLaVA models, general visual features move into the question tokens in lower layers and question-relevant object information in middle layers before reaching the last position [19]; removing object-specific visual tokens lowers object identification by more than 70% [20]. Causal patching separates a direct pathway, read from the image tokens, from a text-mediated pathway through the query tokens, chosen per task, with the second acting as a fallback when the first is ablated [21]. In a hybrid model the text-mediated route has a second carrier: the recurrent state delivers image information to the question tokens without any attention to the image rows. Zeroing the visual KV rows removes the direct pathway in the attention layers and leaves this route intact; zeroing the state does the converse.

The two memories of hybrid models. Afendulev et al. separate the KV cache and the recurrent state of text contexts by keeping one and dropping the other (the convolution cache is dropped with the state) and by swapping states between contexts [11]. We differ in the input (an image span inside the prompt, followed by the question), in the interventions (graded: quantization, error feedback, matched noise and token deletion besides removal) and in the measure (a loss ratio anchored at no-image and full runs). Hybrid video models have used the state as evidence for which visual tokens to keep: as a carrier of discarded tokens [13]

and, through its change over a chunk, as a query-free eviction signal for the KV cache [22].

Linear and hybrid VLMs; recurrent-state compression. Mamba-based VLMs replace the language backbone by a state-space model [23, 24]; there the state is the only memory of the image. InfiniteVL interleaves a small fraction of full-attention layers with Gated DeltaNet in a VLM built by distillation [25]; Qwen3.5 is natively pretrained as a hybrid on text and images [8]. Post-training quantization of the recurrent state of GDN and related layers has been studied for text [26–28], and jointly with the KV cache in [12]. None of these isolates the image.

3 Background

3.1 Qwen3.5 as a hybrid VLM

Qwen3.5 has L decoder layers in repeating groups of three GDN layers and one gated softmax-attention layer, so $L_a = L/4$ attention and $L_g = 3L/4$ GDN layers. For the 4B and 9B models $L = 32$, $L_a = 8$, $L_g = 24$ (Table 1).

Early fusion. A vision encoder with patch size $p = 16$ and $m \times m$ spatial merging ($m = 2$) turns an image resized to $H \times W$ pixels (both multiples of $pm = 32$) into

$$n_v = \frac{H}{p} \cdot \frac{W}{p} \cdot \frac{1}{m^2} = \frac{HW}{(pm)^2} \tag{1}$$

visual tokens, which replace n_v placeholder tokens in the input embedding. In the transformers implementation we use, this scatter at the input embedding is the only place where visual features enter the language model; no visual features are injected at intermediate layers (Appendix A, premise 1). Visual tokens get three-dimensional rotary positions (temporal, height, width; M-RoPE [29]) on the first quarter of each attention head's dimensions. We cap the image at $HW \le 1{,}310{,}720$ pixels, so $n_v \le 1{,}280$.

3.2 Gated DeltaNet

Each GDN head keeps a matrix $S_t \in \mathbb{R}^{d_k \times d_v}$. With L2-normalised key $k_t \in \mathbb{R}^{d_k}$, query $q_t \in \mathbb{R}^{d_k}$ (further scaled by $d_k^{-1/2}$), value $v_t \in \mathbb{R}^{d_v}$, decay $\alpha_t \in (0,1)$ and write strength $\beta_t \in (0,1)$, computed per head from the input at t as $\alpha_t = \exp\big(-e^{A}\,\mathrm{softplus}(a_t + b)\big)$ and $\beta_t = \mathrm{sigmoid}(\beta'_t)$ with a_t, β'_t linear in the input and A, b learned, the state is first decayed and then corrected along k_t:

$$\begin{aligned} S_t &= \alpha_t S_{t-1} + \beta_t\, k_t \big(v_t - \alpha_t S_{t-1}^{\top} k_t\big)^{\top} \\ &= \alpha_t \big(I - \beta_t k_t k_t^{\top}\big) S_{t-1} + \beta_t\, k_t v_t^{\top}, \end{aligned} \tag{2}$$

$$o_t = S_t^{\top} q_t. \tag{3}$$

This is the order of operations of the reference recurrence in the implementation (decay, read $(\alpha_t S_{t-1})^{\top} k_t$, write). Prefill computes the same recurrence in chunks, but the state is written to the cache once per forward call, at its end (premise 5). Before the recurrence, q, k, v pass through a causal depthwise convolution of kernel size $\omega = 4$ whose cache holds the last inputs; we return to it in Section 4.5.

Table 1: Cache geometry, from the model configurations. Sizes by (4); KV in bf16, state in fp32.

Model	L	L_a	L_g	H_{kv}	d	H	d_k=d_v	$\lvert S\rvert$ (MB)
Qwen3.5-4B	32	8	24	4	256	32	128	50.3
Qwen3.5-9B	32	8	24	4	256	32	128	50.3

GDN key heads: 16; value heads $H = 32$; the state is per value head. Vision encoder: patch 16, merge 2; depth 24 (4B), 27 (9B).

3.3 Two memories and their sizes

With H_{kv} KV heads of dimension d per attention layer and H GDN value heads, a sequence of T tokens leaves

$$|\mathrm{KV}|(T) = 2L_a H_{kv} d\, T, \qquad |S| = L_g H d_k d_v \tag{4}$$

elements in the two memories (the factor 2 counts keys and values). The implementation stores the KV cache in the model dtype (bf16, 16 bits) and the recurrent state in fp32 (32 bits; premise 3). For 4B and 9B, $|\mathrm{KV}|(T) = 16{,}384\,T$ and $|S| = 24 \cdot 32 \cdot 128 \cdot 128 = 12{,}582{,}912$, i.e. 32.8 kB of KV per token and a 50.3 MB state. The visual rows of an image therefore occupy $0.0328\,n_v$ MB, at most 41.9 MB at $n_v = 1{,}280$, and the two are equal only at

$$n_v^{\star} = \frac{32\,|S|}{16 \cdot 2L_a H_{kv} d} = 1{,}536 \tag{5}$$

visual tokens. Every image in this study leaves a state larger than its own KV rows. (The crossover of 768 tokens in [12] counts the state in bf16; the implementation used here keeps it in fp32.)

4 Method

4.1 Two-stage prefill and the intervention point

A prompt is the token sequence $x = (P, \mathcal{V}, Q)$: a prefix P (chat template up to and including the vision-start token), the image span $\mathcal{V} = \{s, \dots, e-1\}$ of $n_v = e - s$ placeholder positions carrying the visual features, and the question Q (vision-end token, question, instruction and the assistant header). Let $\mathcal{M}_e$ be the state of all caches after the first e tokens:

$$\mathcal{M}_e = \big(\{K^{\ell}_{1:e}, V^{\ell}_{1:e}\}_{\ell \in \mathcal{A}},\ \{S^{\ell}_e,\ C^{\ell}_e\}_{\ell \in \mathcal{G}}\big), \tag{6}$$

with $\mathcal{A}$, $\mathcal{G}$ the attention and GDN layers and C^{ℓ}_e the convolution cache. Stage 1 runs $(P, \mathcal{V})$ with unaltered computation. An intervention ϕ maps $\mathcal{M}_e$ to $\phi(\mathcal{M}_e)$. Stage 2 runs Q on $\phi(\mathcal{M}_e)$ without further alteration; the reference answer is then scored by teacher forcing on a copy of the resulting cache, and a greedy answer of at most 16 tokens is decoded on the original. Because nothing is altered after the image, any effect of ϕ is an effect on what the image left behind. All conditions, including the unaltered one, use the same two-stage path. [TBD: report the two-stage vs single-pass check on 4B (Colab), not the local 0.8B check.]

4.2 Interventions

KV side. For every attention layer, the rows of the visual positions are replaced, $K_V^\ell \leftarrow f(K_V^\ell)$ and $V_V^\ell \leftarrow f(V_V^\ell)$, with $f \in \{\mathrm{id}, Q_4, N_4, 0\}$. Rows of P are not touched, and rows of Q are written later by stage 2. Keys are cached after the rotary embedding, so f acts on rotated keys.

State side. For every GDN layer, $S_e^\ell \leftarrow g(S_e^\ell)$ with $g \in \{\mathrm{id}, Q_8, Q_6, Q_8^{\mathrm{EF}}, Q_6^{\mathrm{EF}}, N_8, 0\}$. The convolution cache C_e^ℓ is never altered.

Quantizer. Q_b is asymmetric uniform round-to-nearest with one scale and zero point per group of $G = 128$ consecutive elements along the last dimension (the head dimension $d = 256$ for keys and values, two groups per head and token; d_v for the state, one group per row $S[h, i, :]$). For a group $x \in \mathbb{R}^G$,

$$\begin{aligned} \sigma &= \frac{\max x - \min x}{2^b - 1}, \qquad z = \left\lfloor -\min x/\sigma \right\rceil, \\ Q_b(x) &= \sigma\Big(\mathrm{clamp}\big(\lfloor x/\sigma \rceil + z,\ 0,\ 2^b - 1\big) - z\Big), \end{aligned} \tag{7}$$

where $\lfloor\cdot\rceil$ rounds to the nearest integer and σ is floored at 10^{-8}. Quantization is simulated in fp32 and cast back to the cache dtype. Because the grid spans the group's range, every element is within half a step of its stored value, $|Q_b(x)_i - x_i| \le \sigma/2$, i.e. at most $1/(2(2^b-1))$ of the group range: 3.3% at 4 bits, 0.79% at 6 bits and 0.20% at 8 bits.

Matched noise. N_b adds Gaussian noise with the energy of b-bit quantization error but none of its structure. For a cache tensor X (one layer's visual keys, values, or state),

$$N_b(X) = X + \rho_b(X)\, Z, \quad \rho_b(X) = \sqrt{\tfrac{1}{|X|} \sum \big(Q_b(X) - X\big)^2}, \tag{8}$$

with Z standard normal entries of the shape of X. The generator is re-seeded (seed 0) for every tensor, so tensors of the same shape receive the same Z, across layers and samples, scaled by their own ρ_b. The RMS ρ_b is taken over the whole tensor, so noise is spread uniformly where quantization error is concentrated in high-range groups.

Error feedback during the image. Quantizing the state once after the image (Q_b above) commits one rounding error. With error feedback the image is read in chunks of at most 256 visual tokens, with boundaries $s = e_0 < e_1 < \cdots < e_C = e$, $C = \lceil n_v/256 \rceil \le 5$. Let $\hat{S}_{e_u}$ be the state after running chunk u from the stored state $\tilde{S}_{e_{u-1}}$, with $\tilde{S}_{e_0} = S_s$ the unaltered state after the prefix. Then

$$\tilde{S}_{e_u} = Q_b\big(\hat{S}_{e_u} + r_{u-1}\big), \quad r_u = \hat{S}_{e_u} + r_{u-1} - \tilde{S}_{e_u}, \tag{9}$$

with $r_0 = 0$; the final residual r_C is discarded and $\tilde{S}_e$ is what the question reads. Each chunk is a separate forward call, because the state is written once per call (premise 5). For $n_v \le 256$, $C = 1$ and $Q_b^{\mathrm{EF}} = Q_b$. Error feedback here therefore acts across at most five stores, unlike decoding in [12], where it acts across every generated token.

Zeroing. $f = 0$ sets visual keys and values to zero; $g = 0$ sets every GDN state to zero. Section 4.5 states precisely what each removes and what it leaves.

Visual-token deletion. A keep set $I \subset \{0, \ldots, n_v - 1\}$ of size $n_v' = \max(1, \mathrm{round}(\kappa n_v))$, $\kappa \in \{0.5, 0.25\}$, is chosen uniformly, $I = \{\lfloor j\, n_v/n_v' \rfloor : j = 0, \ldots, n_v' - 1\}$, or at random without replacement (seed 0). The other visual tokens are removed from the input before the language model, so neither memory ever sees them; the kept tokens keep their original M-RoPE positions, and the vision encoder still sees the full image. Deletion shrinks the visual KV rows to n_v' and leaves $|S|$ unchanged.

Every-write quantization. For comparison with [12] we also quantize every KV append and every state write of the whole run with the released hooks of that work (scope *whole*); the image and the text are quantized alike. The state is then stored at the end of stage 1, at the end of stage 2, after the teacher-forced pass over the reference answer, and after every decoded token. With error feedback the residual is reset at the first store of each layer (end of stage 1) and carried through the later stores. The residual buffer is shared between the teacher-forced copy of the cache and the decoding cache, so the first decoding step continues from the residual of the teacher-forced pass; this affects the greedy answers of the two every-write cells with error feedback, not their losses. [TBD: fixed in a later tag or reported as is.]

4.3 Retained ratio

Let the reference answer have tokens $a_{1:m}$. For sample i under condition c,

$$\mathrm{NLL}_i(c) = -\frac{1}{m} \sum_{j=1}^{m} \log p_c\big(a_j \mid x,\ a_{<j}\big), \tag{10}$$

where p_c reads the memory produced by condition c. Two anchors use the same prompt: *full* (image, no alteration) and *blind* (no image: the image span and its start and end tokens are removed). With $\bar{N}(c) = \frac{1}{n} \sum_i \mathrm{NLL}_i(c)$ over a task's n samples, the retained ratio is

$$\mathcal{R}(c) = \frac{\bar{N}(\mathrm{blind}) - \bar{N}(c)}{\bar{N}(\mathrm{blind}) - \bar{N}(\mathrm{full})}. \tag{11}$$

$\mathcal{R} = 1$: the altered memory supplies the full reduction in answer loss that the image provides; $\mathcal{R} = 0$: none of it; $\mathcal{R} < 0$: worse than no image. $\mathcal{R}$ is a ratio of dataset means, not a mean of per-sample ratios, whose denominators can be near zero. Writing B_i, F_i, C_i for the three per-sample losses,

$$\begin{aligned} \mathcal{R}(c) &= \frac{\sum_i (B_i - C_i)}{\sum_i (B_i - F_i)} = \sum_i \omega_i \frac{B_i - C_i}{B_i - F_i}, \\ \omega_i &= \frac{B_i - F_i}{\sum_{i'} (B_{i'} - F_{i'})}, \end{aligned} \tag{12}$$

where the second equality requires $B_i \neq F_i$ for every sample. $\mathcal{R}$ thus weights each sample by how much the image

helps it: samples the image barely helps contribute little, and samples it hurts ($B_i < F_i$) enter with negative weight. Every sample's loss is a per-token mean, so long answers do not dominate. It is defined when $\bar{N}(\text{blind}) > \bar{N}(\text{full})$, which we check per task before reporting. The 95% interval is a paired percentile bootstrap [30]: 2,000 resamples of sample indices with replacement (seed 0), the same indices for the three means; resamples with a zero denominator are dropped.

Reference answers and task scores. The reference for the loss is the first answer (DocVQA), the most frequent of the ten annotator answers (TextVQA), or the single answer (ChartQA, POPE). Greedy answers are scored with ANLS at $\tau = 0.5$ [31] (DocVQA), relaxed accuracy with 5% relative tolerance [32, 33] (ChartQA), VQA accuracy $\frac{1}{10}\sum_{i=1}^{10} \min\big(1, \frac{1}{3}\#\{j \neq i : a^{(j)} = \hat{a}\}\big)$ [34] (TextVQA), and yes/no accuracy and F1 with *yes* as the positive class (POPE), all after one shared normalisation (lower case; punctuation except decimal points and percent signs removed; articles removed).

4.4 Memory accounting

For the equal-memory comparison we charge a condition

$$\mathrm{MB}(c) = \frac{b_{kv}\,|\mathrm{KV}|(n'_v + n_t) + b_s\,|S|}{8 \cdot 10^6}, \qquad (13)$$

with n'_v the visual tokens kept, n_t the text tokens, $b_{kv} \in \{16, 4, 0\}$ (full, 4-bit, zero) and $b_s \in \{32, 8, 6, 0\}$; noise is charged at its matched bit-width. This is deployment accounting: all KV rows are charged at b_{kv}, although the one-shot intervention leaves the text rows at 16 bits. Scales and zero points (about $32/G = 0.25$ bits per element for fp16 pairs) and the convolution cache are not counted.

4.5 Caveats of the interventions

Zeroed keys still take attention. A zero key gives the logit $q^\top 0/\sqrt{d} = 0$, not $-\infty$, so each zeroed visual position keeps a softmax weight $w_i = 1/\mathcal{Z}$, with $\mathcal{Z}$ the softmax normaliser of the query, and contributes the zero value. The attention output becomes $\sum_{i \notin \mathcal{V}} w_i v_i$ with $\sum_{i \notin \mathcal{V}} w_i = 1 - n_v/\mathcal{Z} < 1$: KV zero removes the visual content and also shrinks the attention output by the mass the zeroed rows absorb. Masking the visual positions would remove the content and renormalise; [11] avoid zero keys for this reason. [TBD: decide whether to add a KV-drop condition (visual rows removed from the cache); until then KV zero is read as removal plus attention dilution.]

The convolution cache leaks the last visual tokens. Unlike [11], who drop the convolution cache together with the state, we leave it in place, following [12]. C_e^ℓ holds the inputs of the last positions of the span, and the causal convolution of kernel $\omega = 4$ mixes them into the GDN inputs of the first $\omega - 1 = 3$ tokens after the span. State zero therefore leaves a short local channel from the last three visual tokens into the first question-side tokens (and, through their hidden states, into everything above them).

Table 2: Carriers of image information after each intervention. ✓ intact, ~ degraded, – removed. Conv: the last three visual tokens through the convolution cache.

Intervention	visual KV	state	conv
full	✓	✓	✓
KV Q_4, N_4	~	✓	✓
KV 0	– (dilution)	✓	✓
state Q_b, Q_b^{EF}, N_8	✓	~	✓
state 0	✓	–	✓
KV 0 + state 0	– (dilution)	–	✓
deletion, keep κ	κn_v rows	κn_v tokens	✓

What each intervention leaves. Table 2 lists the carriers of image information left after each intervention. The prefix rows of the KV cache precede the image and carry none of it. Zeroing both memories leaves only the convolution channel and the attention dilution of the zeroed keys, so $\mathcal{R}$ of that cell bounds what these two side effects contribute.

State zero also forgets the prefix. S_e^ℓ summarises P as well as $\mathcal{V}$; zeroing it removes both. The prefix is the fixed chat-template header, the same for every sample.

4.6 Predictions

We state what prior work predicts before reporting results. (P1) If exact retrieval goes through attention, as in text [11], then reading tasks (DocVQA, TextVQA, and the values read off charts in ChartQA) should lose most of the image's contribution under KV zero and little under state zero. (P2) A coarse judgement such as object presence (POPE) may survive through the state alone, as the gist of a text context does [11]. (P3) In the one-shot cells a single rounding error is injected after the image and then carried, and damped by the decay, through the unquantized updates of the question tokens; the repeated injection that made the state fragile under every-write quantization [12] does not occur, so their loss should be well below that of the every-write cells at the same bit-widths. (P4) Error feedback can only matter for images with $n_v > 256$ ($C > 1$). [TBD: keep, revise or drop each prediction against the results; report P1–P4 outcomes explicitly.]

5 Experimental Setup

Models. Qwen3.5-4B (phases 1–3) and Qwen3.5-9B (phase 4), bf16 weights, the post-trained (non-base) checkpoints. GDN runs in the library's PyTorch kernels. [TBD: confirm from the Colab environment that no fla kernel path was taken, and report torch/transformers versions recorded in the result rows.]

Data. Fixed random subsets (seed 0) of DocVQA validation (200) [35], ChartQA test (200) [33], TextVQA validation (200) [36] and POPE (300) [37], from the lmms-lab copies used by lmms-eval [38] at pinned revisions; the sample ids are released. Every question gets the same instruction, "Answer the question using a single word or

phrase.", in the default chat template without a system prompt; the template closes an empty thinking block before the answer (non-thinking mode).

Conditions. Phase 1 (which memory): $f \in \{\mathrm{id}, Q_4, N_4, 0\}$ with the state unaltered; $g \in \{Q_8, Q_8^{\mathrm{EF}}, Q_6, Q_6^{\mathrm{EF}}, N_8, 0\}$ with the KV unaltered; and both zeroed. Phase 2 (bits): the state levels alone and with $f = Q_4$, and the two every-write cells (KV4 with Q_8 and Q_8^{EF}). Phase 3 (tokens versus bits): $\kappa \in \{1, 0.5, 0.25\}$ uniform, with KV at 16 or 4 bits, and $\kappa = 0.5$ random. Phase 4 (9B): full, KV zero, state zero, KV4 with Q_8^{EF}, and $\kappa = 0.5$ uniform. Each condition also has the two anchors per task. Batch size 1 and greedy decoding throughout.

Compute. One NVIDIA A100-SXM4-80GB on Google Colab. A 4B preflight (22 conditions × 4 tasks × 5 samples) took 1.47 s per sample on average, with a peak of 9.3 GB of GPU memory (Appendix C). [TBD: total GPU hours and compute units per phase.]

6 Results

6.1 Which memory carries the image

[TBD: Figure: retained per condition and task with 95% CIs (paper/figures/retained_Qwen3.5-4B.pdf). Table: phase-1 retained. Text: KV zero vs state zero; KV noise vs KV4; state noise vs state 8 (structure vs energy, cf. the noise control in [12]); both zeroed versus blind (should be near 0 unless the conv leak carries information).]

6.2 How many bits each memory needs after reading

[TBD: State 8/6 with and without EF, alone and with KV4; P3 (one-shot versus every-write at KV4 with Q_8 and Q_8^{EF}); P4 (split EF effects by C, with the n_v distribution per task from the result rows).]

6.3 Fewer visual tokens or fewer bits

The image's KV rows cost $\kappa\, b_{kv}$ bits per element and original visual token: 16, 8, 4 at $\kappa = 1, 0.5, 0.25$ with 16-bit KV, and 4, 2, 1 with 4-bit KV. The equal-memory pair is therefore $\kappa = 0.25$ at 16 bits against $\kappa = 1$ at 4 bits (4 bits each); the state (32 bits) is the same in both, and the text rows differ (16 versus 4 bits), which (13) charges. [TBD: retained and task scores of the pair; the full $\kappa \times b_{kv}$ grid as a frontier against MB; uniform versus random at $\kappa = 0.5$.]

6.4 Scale

[TBD: 9B vs 4B on the five phase-4 conditions.]

6.5 Task scores

[TBD: ANLS / relaxed accuracy / VQA accuracy / POPE F1 per condition; does the ordering agree with $\mathcal{R}$?]

7 Discussion and Limitations

[TBD: Practical reading for pruning and KV compression of hybrid VLMs.]

Limitations. (i) KV zero keeps softmax mass on the zeroed rows and state zero leaves the convolution cache (Section 4.5); neither is a perfect deletion. (ii) Each intervention acts once, after the image; it does not model a cache that stays quantized while the answer is generated, except for the two every-write cells. (iii) $\mathcal{R}$ is measured with the loss of one reference answer per sample; the task scores check that it tracks answer quality, but a reference that the model would phrase differently inflates both anchors. (iv) Removing a pathway can show what the model is able to fall back on rather than what it normally uses [21]: $\mathcal{R}$ under KV zero or state zero measures the information the other memory can supply on its own, not how much of it the intact model draws from each. (v) Subsets of 200–300 samples per task, one model family at two sizes, one image resolution cap. (vi) Quantization is simulated; no memory or speed is saved in these runs.

8 Conclusion

[TBD: after results.]

Acknowledgments

This work was carried out by a single author without institutional funding; compute was self-funded on Google Colab. The author used Claude (Anthropic) as an assistant for code development and manuscript editing; all experiments, analyses and claims were designed, run and verified by the author.

Reproducibility and Data Availability

Code, configurations, subset ids, raw per-sample results and the scripts that regenerate every table and figure are at `https://github.com/skytinstone/twomemories` [TBD: make public at submission]. Each phase is run by `python scripts/run_phase.py --phase <name>`, the tables by `python analysis/make_tables.py` and the figures by `python analysis/figures.py`. Quantization uses the released `hybridquant` package [12] unchanged. All datasets are public.

References

[1] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2403.06764.

[2] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2410.04417.

[3] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. arXiv:2403.15388.

[4] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19792–19802, 2025. arXiv:2412.04467.

[5] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, and Dahua Lin. PyramidDrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2410.17247.

[6] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. LOOK-M: Look-once optimization in KV cache for efficient multimodal long-context inference. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4065–4078, 2024. arXiv:2406.18139.

[7] Dezhan Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. VL-Cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.23317.

[8] Qwen Team. Qwen3.5: Towards native multimodal agents. `https://qwen.ai/blog?id=qwen3.5`, February 2026.

[9] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.

[10] vLLM contributors. [rfc]: fp8 kv cache gives little to no memory benefit on mamba/gdn hybrid models. `https://github.com/vllm-project/vllm/issues/55196`, 2026. VERIFY: issue number and the GSM8K claim quoted in the text.

[11] Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. What attention recalls and recurrence controls in hybrid language models. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2609.04434.

[12] Minseok Shin. Joint KV-cache and recurrent-state quantization in hybrid linear-attention LLMs, 2026. Unpublished manuscript; code at `https://github.com/skytinstone/hybridquant`.

[13] Jindong Jiang, Amala Sanjay Deshmukh, Kateryna Chumachenko, Karan Sapra, Zhiding Yu, Guilin Liu, Andrew Tao, Pavlo Molchanov, Jan Kautz, and Wonmin Byeon. Stateful token reduction for long-video hybrid VLMs. *arXiv preprint arXiv:2603.00198*, 2026.

[14] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.09461.

[15] Seong Yeol Park, Min Jung Kwon, Xianghua Piao, and Yeong Hyeon Gu. DTP: Delta-guided two stage pruning for Mamba-based multimodal large language models. In *International Conference on Learning Representations (ICLR)*, 2026.

[16] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *ICML*, 2024. arXiv:2402.02750.

[17] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *NeurIPS*, 2024. arXiv:2401.18079.

[18] Omri Kaduri, Shai Bagon, and Tali Dekel. What's in the image? A deep-dive into the vision of vision language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2411.17491.

[19] Zhi Zhang, Srishti Yadav, Fengze Han, and Ekaterina Shutova. Cross-modal information flow in multimodal large language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2411.18620.

[20] Clement Neo, Luke Ong, Philip Torr, Mor Geva, David Krueger, and Fazl Barez. Towards interpreting visual information processing in vision-language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.07149.

[21] Israfel Salazar, Stella Frank, Dan Oneata, Desmond Elliott, and Constanza Fierro. Pathways of visual information flow in vision-language models. *arXiv preprint arXiv:2607.03358*, 2026.

[22] Taeyoun Kwon, Seungjin Kim, Hyeonyu Kim, and Moon Hwan Kim. DeltaS: Reading the gated linear attention state for KV cache eviction in streaming video. *arXiv preprint arXiv:2609.27470*, 2026.

[23] Han Zhao, Min Zhang, Wei Zhao, Pengxiang Ding, Siteng Huang, and Donglin Wang. Cobra: Extending Mamba to multi-modal large language model for efficient inference. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2025. arXiv:2403.14520.

[24] Yanyuan Qiao, Zheng Yu, Longteng Guo, Sihan Chen, Zijia Zhao, Mingzhen Sun, Qi Wu, and Jing Liu. VL-Mamba: Exploring state space models for multimodal learning. In *NeurIPS Efficient Natural Language and Speech Processing Workshop (ENLSP-IV), PMLR 262*, pages 102–113, 2024. arXiv:2403.13600.

[25] Hongyuan Tao, Bencheng Liao, Shaoyu Chen, Haoran Yin, Qian Zhang, Wenyu Liu, and Xinggang Wang. InfiniteVL: Synergizing linear and sparse attention for highly-efficient, unlimited-input vision-language models. *arXiv preprint arXiv:2512.08829*, 2025.

[26] Tao Zhang, Jianchao Tan, Pingwei Sun, Yanqi Yu, Zunhai Su, Zixu Jiang, Yuchen Xie, Xunliang Cai, and Ziqian Zeng. Damp: Decay-aware mixed-precision recurrent-state quantization. *arXiv preprint arXiv:2608.27513*, 2026.

[27] Bingchen Yao, Haobo Xu, Haokun Lin, Yichen Wu, Ziyu Guo, Renrui Zhang, Zhichao Lu, Zhenan Sun, and Ying Wei. Stepquant: When and where errors matter in delta-rule recurrent state quantization. *arXiv preprint arXiv:2609.38169*, 2026.

[28] Yi Pan, Haocheng Xi, Kan Zhu, Xingyang Li, Yibo Wu, Mayank Mishra, Hongtao Zhang, William X. Zheng, Baris Kasikci, Song Han, Kurt Keutzer, Rishabh Iyer, and Ion Stoica. Leapquant: Efficient linear attention with accurate recurrent state quantization. *arXiv preprint arXiv:2609.38166*, 2026.

[29] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[30] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman & Hall/CRC, 1993.

[31] Ali Furkan Biten, Ruben Tito, Andres Mafla, Lluis Gomez, Marçal Rusiñol, Ernest Valveny, C. V. Jawahar, and Dimosthenis Karatzas. Scene text visual question answering. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1905.13648.

[32] Nitesh Methani, Pritha Ganguly, Mitesh M. Khapra, and Pratyush Kumar. PlotQA: Reasoning over scientific plots. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020. arXiv:1909.00997.

[33] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. arXiv:2203.10244.

[34] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. VQA: Visual question answering. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2425–2433, 2015. arXiv:1505.00468.

[35] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A dataset for VQA on document images. In *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2021. arXiv:2007.00398.

[36] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards VQA models that can read. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. arXiv:1904.08920.

[37] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.10355.

[38] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. LMMs-Eval: Reality check on the evaluation of large multimodal models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 881–916, 2025. arXiv:2407.12772.

A Implementation premises

Checked against transformers 5.22.2 (Qwen3.5 modeling and cache code) before any run; file and line references are in `docs/assumptions.md` of the repository.

	Premise	Consequence for the method
1	Visual features are scattered into the input embedding only; no intermediate-layer injection.	The caches after the span (KV, state, convolution cache) are the only carriers of the image.
2	The image is one contiguous span of placeholder tokens of length $\prod(\text{grid})/m^2$.	$\mathcal{V}$ is known exactly; (1).
3	The recurrent state is stored in fp32 (PyTorch path).	State charged at 32 bits; (5). [TBD: GPU path].
4	The cache is a `DynamicCache` with attention and linear-attention layers; the write methods hooked by `hybridquant` exist unchanged.	No adapter; every-write cells use the hooks as released.
5	Prefill computes GDN in 64-token chunks internally but writes the state once per forward call.	Error feedback during the image needs one forward call per 256-token chunk (9).

B Conditions

All 22 conditions on Qwen3.5-4B, with the phases that run them; phase 4 runs the five marked P4 on Qwen3.5-9B. Bits are the charges of (13). Every task also has the blind anchor.

Label	KV f	state g	tokens	scope	phases	b_{kv}	b_s
KVfull_Sfull	id	id	all	image	P1–P4	16	32
KV4_Sfull	Q_4	id	all	image	P1, P2, P3	4	32
KVnoise_Sfull	N_4	id	all	image	P1	4	32
KVzero_Sfull	0	id	all	image	P1, P4	0	32
KVfull_S8	id	Q_8	all	image	P1, P2	16	8
KVfull_S8ef	id	Q_8^{EF}	all	image	P1, P2	16	8
KVfull_S6	id	Q_6	all	image	P1, P2	16	6
KVfull_S6ef	id	Q_6^{EF}	all	image	P1, P2	16	6
KVfull_Snoise	id	N_8	all	image	P1	16	8
KVfull_Szero	id	0	all	image	P1, P4	16	0
KVzero_Szero	0	0	all	image	P1	0	0
KV4_S8	Q_4	Q_8	all	image	P2	4	8
KV4_S8ef	Q_4	Q_8^{EF}	all	image	P2, P4	4	8
KV4_S6	Q_4	Q_6	all	image	P2	4	6
KV4_S6ef	Q_4	Q_6^{EF}	all	image	P2	4	6
KV4_S8_whole	Q_4	Q_8	all	whole	P2	4	8
KV4_S8ef_whole	Q_4	Q_8^{EF}	all	whole	P2	4	8
KVfull_Sfull_T50u	id	id	50% uniform	image	P3, P4	16	32
KV4_Sfull_T50u	Q_4	id	50% uniform	image	P3	4	32
KVfull_Sfull_T25u	id	id	25% uniform	image	P3	16	32
KV4_Sfull_T25u	Q_4	id	25% uniform	image	P3	4	32
KVfull_Sfull_T50r	id	id	50% random	image	P3	16	32

C Compute

One NVIDIA A100-SXM4-80GB (Google Colab, 8.88 compute units per hour as shown by the resource panel). Seconds are summed per-sample wall time of the evaluation (two-stage prefill, teacher-forced pass, greedy decoding), excluding model loading and data download.

Run	Model	samples	s/sample	peak GPU GB	units
preflight (22 conditions × 4 tasks × 5)	4B	455	1.47	9.3	1.64
phase 1	4B	[TBD:]			
phase 2	4B	[TBD:]			
phase 3	4B	[TBD:]			
phase 4	9B	[TBD:]			

[TBD: the preflight row counts the 91 compute-log cells that were recorded (one cell's record was lost in a first, interrupted attempt); recompute from compute_log.jsonl at the end.]

D Full results

[TBD: generated by analysis/make_tables.py from results/**/*.jsonl.]